

Computer Architecture: Layers of Programs

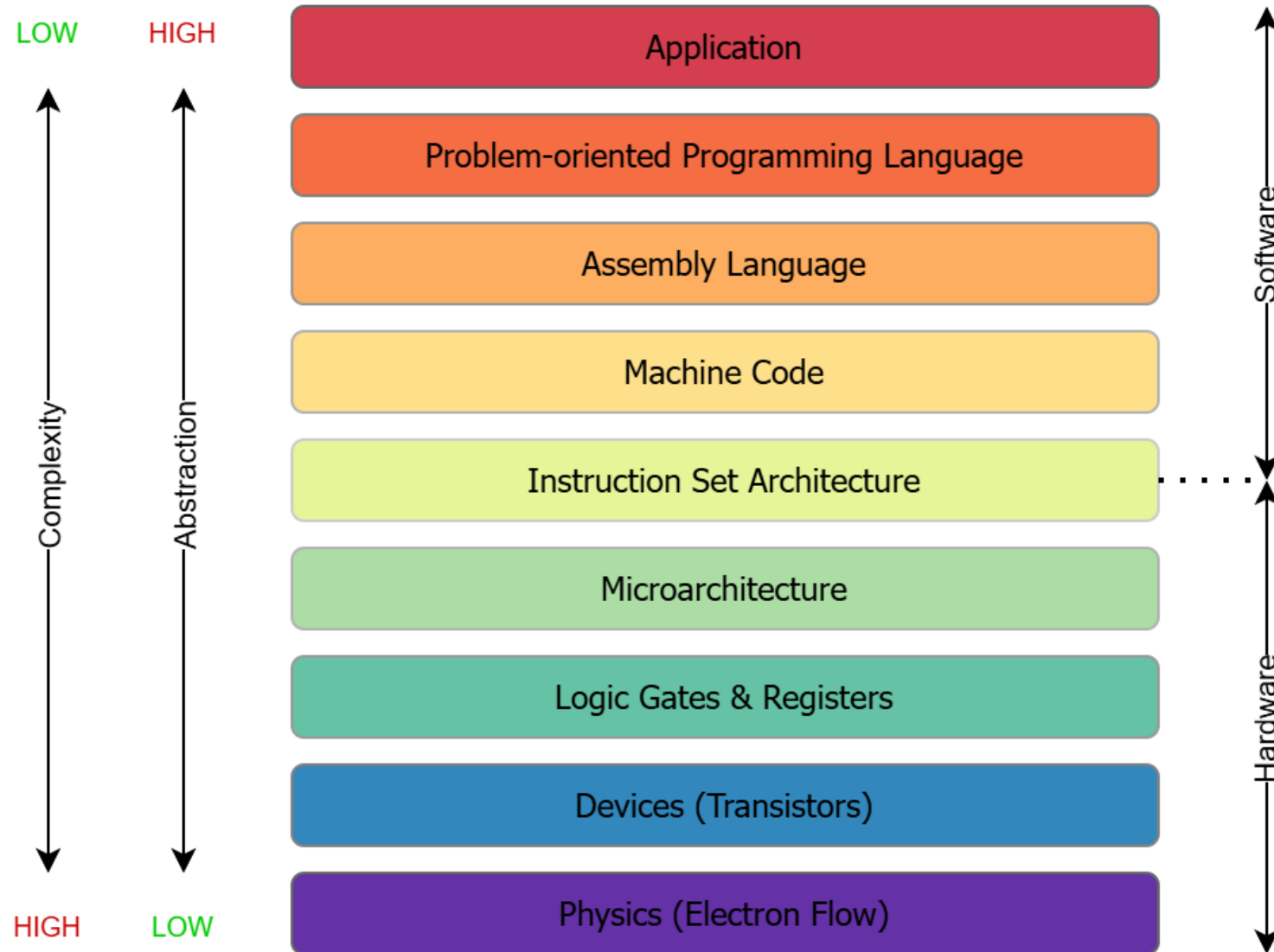
Dr. Colin O'Flynn

e.g., A Computer Program

```
#include <stdio.h>

int main(void) {
    printf("Hello World\n");
    return 0;
}
```

Layers of Abstraction



Considerations Between Layers

- Each layer makes assumptions in both directions.
- Failing to understand these assumptions & limitations causes:
 - Bugs
 - Security flaws
 - Poor performance (or at least not *maximum possible* performance)
 - Code that *doesn't even work*

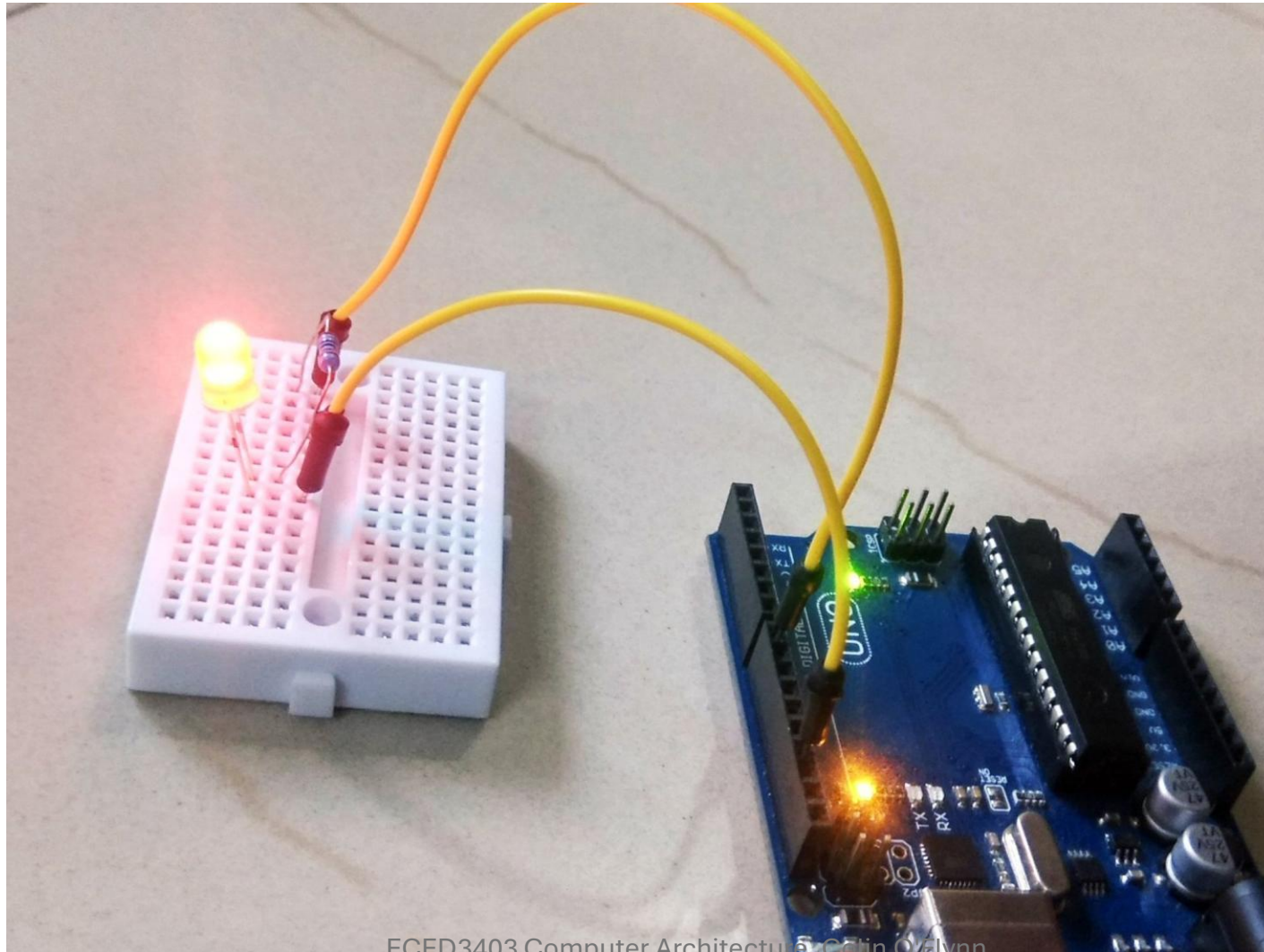
SIDEBAR: “Compiler Explorer” (Godbolt.org)

The screenshot displays the Compiler Explorer (Godbolt.org) interface. The left pane shows the C++ source code for a program with two functions, `gpio_high` and `gpio_low`, and a `test` function that calls them in a loop. The right pane shows the generated ARM assembly for the `test` function, compiled with ARM GCC 14.1.0. The assembly includes instructions for pushing registers, branching to `.L3`, calling `gpio_high` and `gpio_low`, loading a constant from `.L6`, and performing a subtraction and branch to `.L2`. A label `.L6` is defined at the bottom with the value 5000.

```
1  int gpio_high(void);
2  int gpio_low(void);
3
4  void test(void)
5  {
6      while(1){
7          gpio_high();
8          for(int i = 0; i < 5000; i++);
9          gpio_low();
10         for(int i = 0; i < 5000; i++);
11     }
12 }
```

```
1  test():
2      push    {r4, lr}
3  .L3:
4      bl      gpio_high()
5      bl      gpio_low()
6      ldr     r3, .L6
7  .L2:
8      subs    r3, r3, #1
9      bne     .L2
10     b       .L3
11  .L6:
12     .word   5000
```

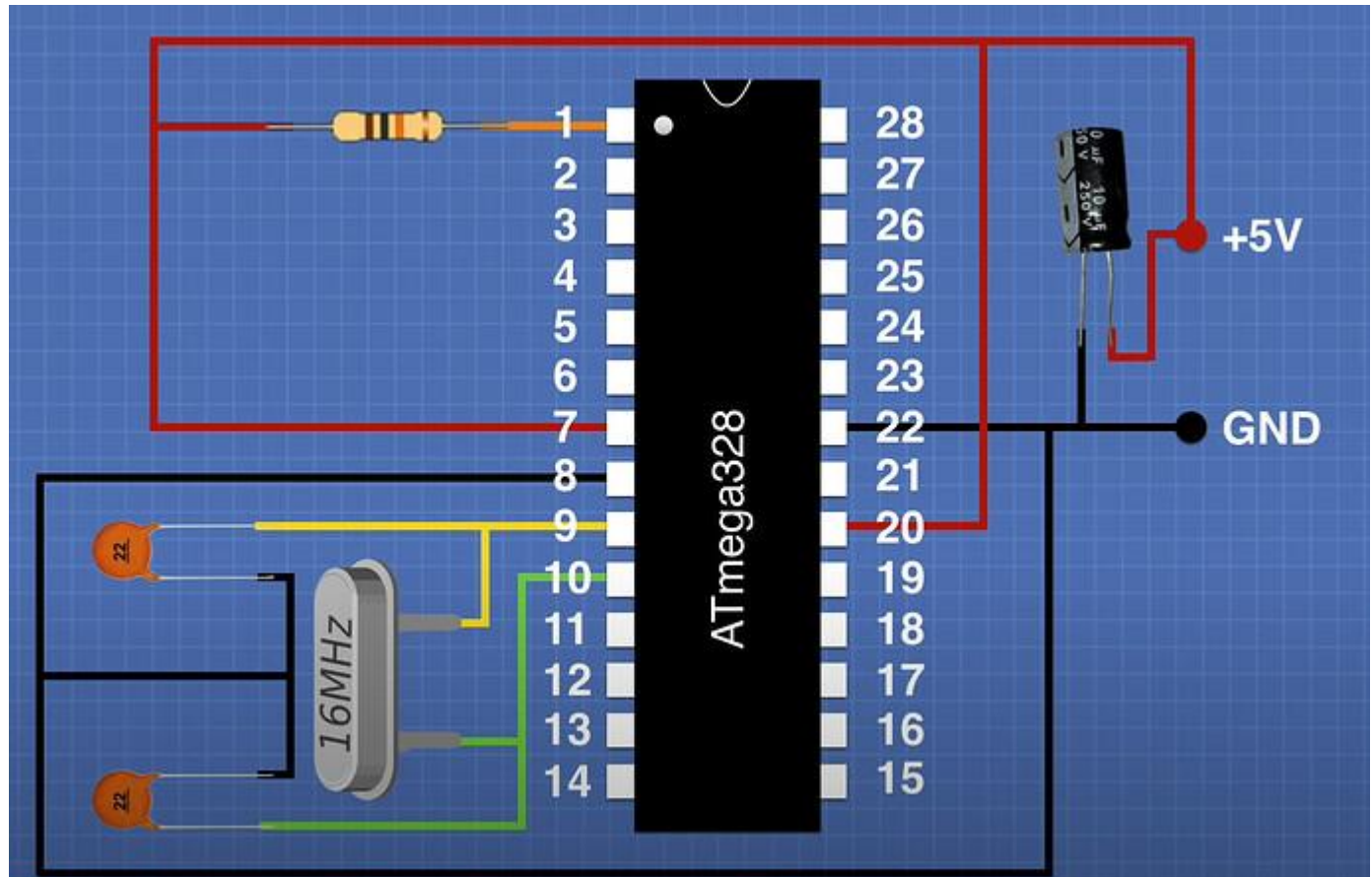
Example #1 : Blinking an LED (Application)



Example #1: Blinking an LED (High Level Code)

```
int gpio_high(void);
int gpio_low(void);

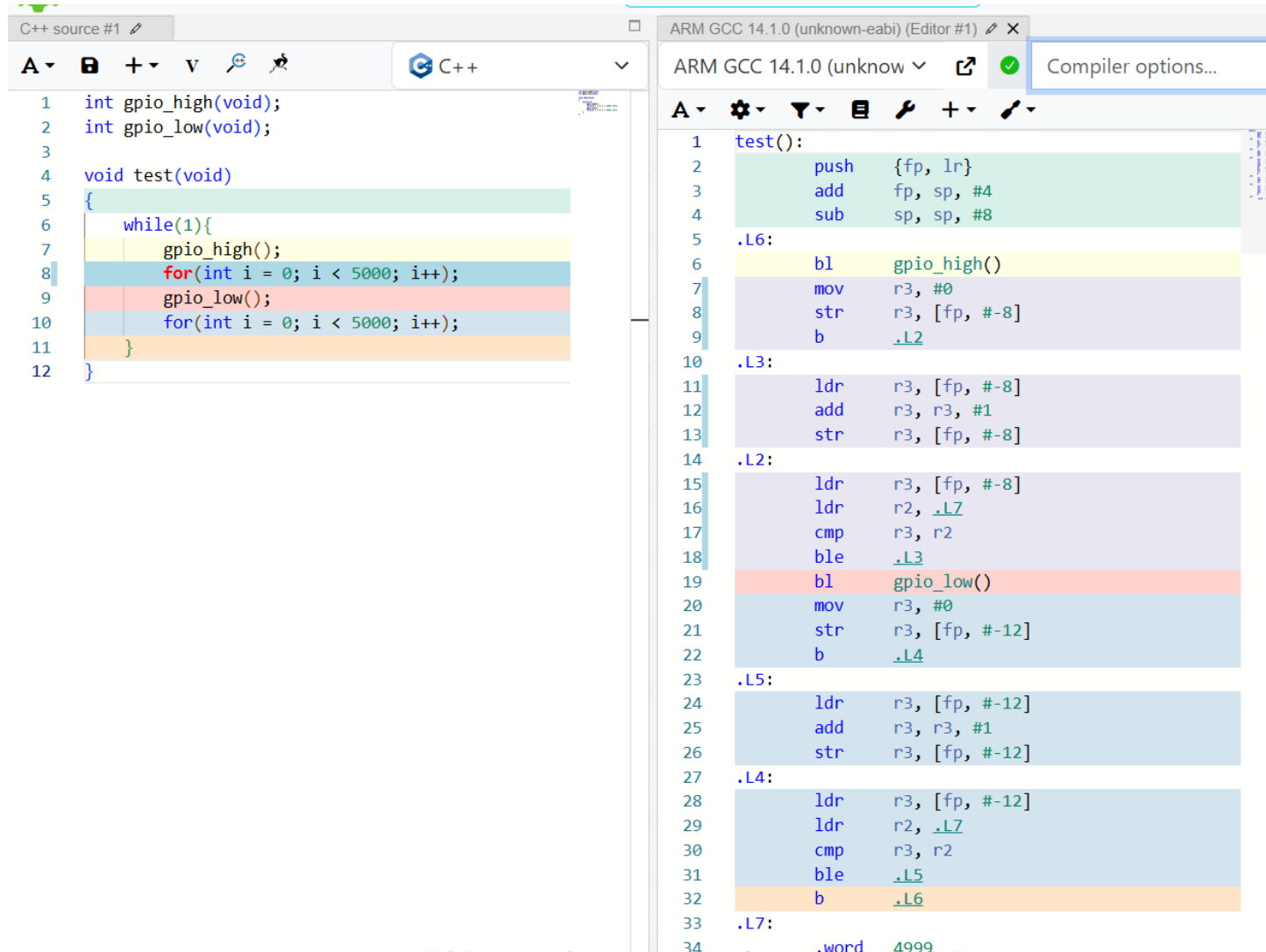
void test(void)
{
    while(1){
        gpio_high();
        _delay_ms(500);
        gpio_low();
        _delay_ms(500);
    }
}
```



Example #1: Delays in C on a Microcontroller

```
int gpio_high(void);  
int gpio_low(void);  
  
void test(void)  
{  
    while(1){  
        gpio_high();  
        for(int i = 0; i < 5000; i++);  
        gpio_low();  
        for(int i = 0; i < 5000; i++);  
    }  
}
```


(Backup Slides – Skipped if Live Works)



The image shows a side-by-side comparison of C++ source code and its compiled ARM assembly. The left pane, titled 'C++ source #1', contains the following code:

```
1 int gpio_high(void);
2 int gpio_low(void);
3
4 void test(void)
5 {
6     while(1){
7         gpio_high();
8         for(int i = 0; i < 5000; i++);
9         gpio_low();
10        for(int i = 0; i < 5000; i++);
11    }
12 }
```

The right pane, titled 'ARM GCC 14.1.0 (unknown-eabi) (Editor #1)', shows the corresponding assembly code:

```
1 test():
2     push    {fp, lr}
3     add     fp, sp, #4
4     sub     sp, sp, #8
5     .L6:
6     bl      gpio_high()
7     mov     r3, #0
8     str     r3, [fp, #-8]
9     b       .L2
10    .L3:
11    ldr     r3, [fp, #-8]
12    add     r3, r3, #1
13    str     r3, [fp, #-8]
14    .L2:
15    ldr     r3, [fp, #-8]
16    ldr     r2, .L7
17    cmp     r3, r2
18    ble     .L3
19    bl      gpio_low()
20    mov     r3, #0
21    str     r3, [fp, #-12]
22    b       .L4
23    .L5:
24    ldr     r3, [fp, #-12]
25    add     r3, r3, #1
26    str     r3, [fp, #-12]
27    .L4:
28    ldr     r3, [fp, #-12]
29    ldr     r2, .L7
30    cmp     r3, r2
31    ble     .L5
32    b       .L6
33    .L7:
34    .word   4999
```

(Backup Slides – Skipped if Live Works)

The image shows a side-by-side comparison of C++ source code and its compiled ARM assembly. On the left, the C++ source code defines two functions, `gpio_high` and `gpio_low`, and a `test` function that calls them in a loop. On the right, the ARM assembly output for the `test` function is shown, including instructions for pushing registers, branching to `gpio_high` and `gpio_low`, and a loop counter. A red box highlights the `-O1` optimization flag in the assembly editor's toolbar.

```
C++ source #1
1 int gpio_high(void);
2 int gpio_low(void);
3
4 void test(void)
5 {
6     while(1){
7         gpio_high();
8         for(int i = 0; i < 5000; i++);
9         gpio_low();
10        for(int i = 0; i < 5000; i++);
11    }
12 }
```

```
ARM GCC 14.1.0 (unknown-eabi) (Editor #1)
ARM GCC 14.1.0 (unknow
-O1
Output... Filter... Libraries Overrides + Add new... Add tool...
1 test():
2     push    {r4, lr}
3 .L4:
4     bl      gpio_high()
5     ldr     r3, .L8
6 .L2:
7     subs    r3, r3, #1
8     bne     .L2
9     bl      gpio_low()
10    ldr     r3, .L8
11 .L3:
12    subs    r3, r3, #1
13    bne     .L3
14    b       .L4
15 .L8:
16    .word    5000
```

... that is a little easier to understand!

(Backup Slides – Skipped if Live Works)

The image shows a side-by-side comparison of C++ source code and its compiled ARM assembly. On the left, the C++ source code is in a file named 'C++ source #1'. It defines two functions, `gpio_high(void)` and `gpio_low(void)`, and a `test(void)` function. The `test` function contains a `while(1)` loop with two `for` loops, each iterating 5000 times and calling `gpio_high()` and `gpio_low()` respectively. Line 8 of the C++ code is `for(int i = 0; i < 5000; i++);` inside the first `for` loop of the `while` loop.

On the right, the ARM assembly output is shown for 'ARM GCC 14.1.0 (unknown-eabi)'. The assembly code for the `test()` function is displayed. It starts with a `push {r4, lr}` instruction, followed by a label `.L3:`. The assembly then branches to `gpio_high()` and `gpio_low()`. A label `.L2:` is used for a loop decrement and branch. A label `.L6:` is used for a branch instruction. The assembly ends with a `.word 5000` instruction. A red box highlights the `-Os` optimization flag in the compiler settings, indicating that the code is optimized for size.

... What happened to line 8 on the left !?

(Backup Slides – Skipped if Live Works)

```
C++ source #1
1  int gpio_high(void);
2  int gpio_low(void);
3
4  void test(void)
5  {
6      while(1){
7          gpio_high();
8          for(int i = 0; i < 5; i++);
9          gpio_low();
10         for(int i = 0; i < 5; i++);
11     }
12 }
```

```
ARM GCC 14.1.0 (unknown-eabi) (Editor #1)
ARM GCC 14.1.0 (unknown-eabi) (Editor #1) -Os
1  test():
2      push    {r4, lr}
3  .L2:
4      bl     gpio_high()
5      bl     gpio_low()
6      b      .L2
```

...Where did the delays go at all?

(Backup Slides – Skipped if Live Works)

The image shows a side-by-side comparison of C++ source code and its compiled assembly. On the left, the C++ source code defines two functions, `gpio_high` and `gpio_low`, and a `test` function. The `test` function contains a `while(1)` loop with two `for` loops, each calling `gpio_high` and `gpio_low` five times. On the right, the assembly code for the `test` function is shown, generated by ARM GCC 14.1.0. The assembly consists of a `test` function prologue (pushing `r4` and `lr`), followed by a loop labeled `.L2` containing `bl gpio_high()`, `bl gpio_low()`, and a `b .L2` instruction. Two red arrows originate from the `for` loops in the C++ code and point to the corresponding assembly instructions, highlighting the absence of any delay instructions in the compiled output.

```
C++ source #1
1 int gpio_high(void);
2 int gpio_low(void);
3
4 void test(void)
5 {
6     while(1){
7         gpio_high();
8         for(int i = 0; i < 5; i++);
9         gpio_low();
10        for(int i = 0; i < 5; i++);
11    }
12 }
```

```
ARM GCC 14.1.0 (unknown-eabi) (Editor #1)
ARM GCC 14.1.0 (unknow -Os
1 test():
2     push    {r4, lr}
3 .L2:
4     bl      gpio_high()
5     bl      gpio_low()
6     b       .L2
```

...Where did the delays go at all?

(Backup Slides – Skipped if Live Works)

The image shows a side-by-side comparison of C++ source code and its compiled ARM assembly. On the left, the C++ source file 'C++ source #1' contains a function `test(void)` with a `while(1)` loop. Inside the loop, there are two calls to `gpio_high()` and `gpio_low()`, each followed by a `for` loop that iterates 5 times over a `volatile int i`. The `volatile` keyword is highlighted in red in the original image, and a red arrow points from it to the assembly output on the right. On the right, the ARM assembly output for 'ARM GCC 14.1.0 (unknown-eabi)' is shown. The assembly code for the `test()` function includes instructions like `push {r0, r1, r4, lr}`, `mov r4, #0`, and several `bl` (branch with link) instructions to `gpio_high()` and `gpio_low()`. The assembly code is color-coded to match the C++ source: yellow for `gpio_high()` calls and blue for `gpio_low()` calls. The `volatile` keyword in the C++ code is highlighted in red, and a red arrow points from it to the assembly output, indicating that the compiler uses the `volatile` keyword to generate specific assembly instructions that prevent optimization of the variable `i`.

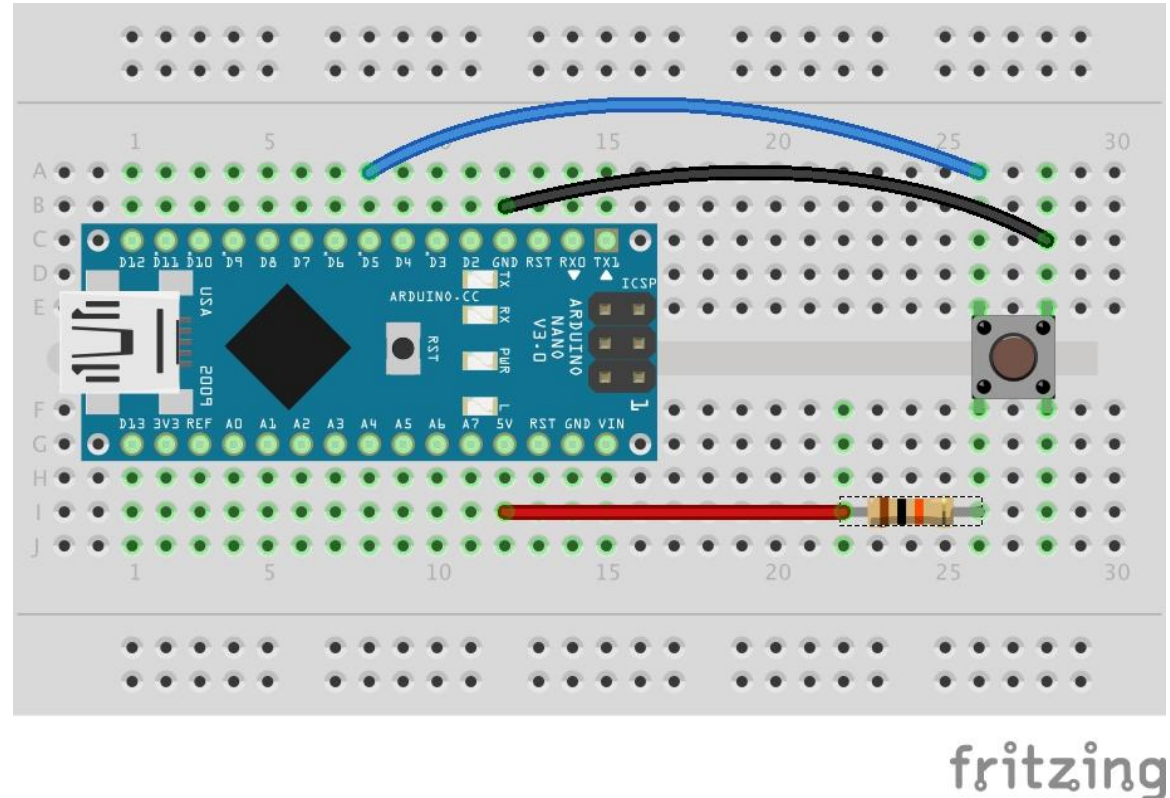
```
C++ source #1
1 int gpio_high(void);
2 int gpio_low(void);
3
4 void test(void)
5 {
6     while(1){
7         gpio_high();
8         for(volatile int i = 0; i < 5; i++);
9         gpio_low();
10        for(volatile int i = 0; i < 5; i++);
11    }
12 }
```

```
ARM GCC 14.1.0 (unknown-eabi) (Editor #1)
ARM GCC 14.1.0 (unknown-eabi) -Os
1 test():
2     push    {r0, r1, r4, lr}
3     mov     r4, #0
4 .L6:
5     bl      gpio_high()
6     str     r4, [sp]
7 .L2:
8     ldr     r3, [sp]
9     cmp     r3, #4
10    ble     .L3
11    bl      gpio_low()
12    str     r4, [sp, #4]
13 .L4:
14    ldr     r3, [sp, #4]
15    cmp     r3, #4
16    bgt     .L6
17    ldr     r3, [sp, #4]
18    add     r3, r3, #1
19    str     r3, [sp, #4]
20    b       .L4
21 .L3:
22    ldr     r3, [sp]
23    add     r3, r3, #1
24    str     r3, [sp]
25    b       .L2
```

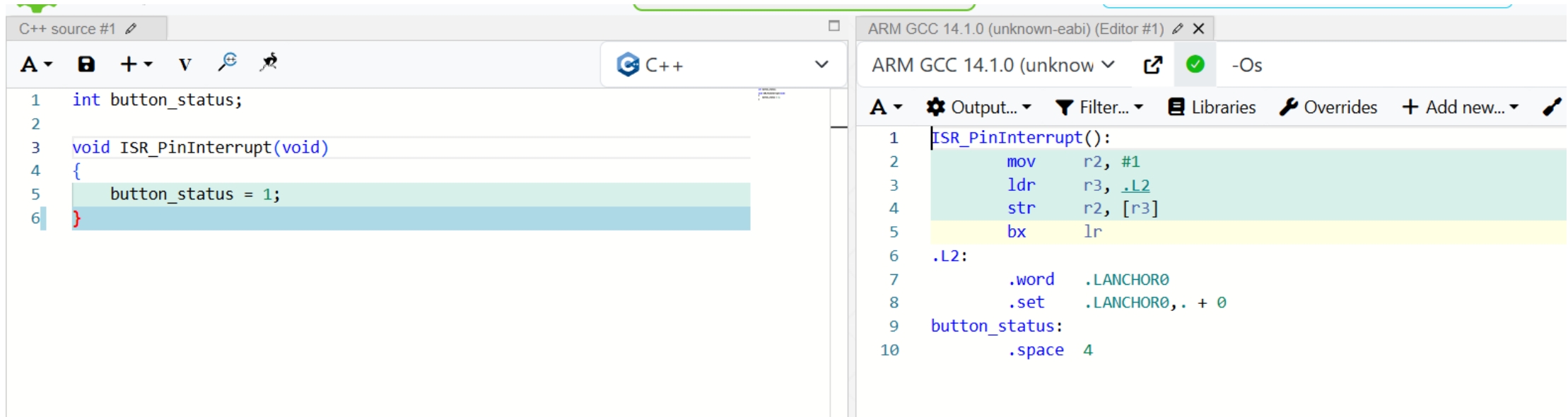
This tells the compiler not to touch my variable.

Example #2: Wait for Button Press

```
int button_status;  
  
void wait_for_button_press(void)  
{  
    button_status = 0;  
  
    while(button_status == 0) {  
        ;  
    }  
}  
  
void ISR_PinInterrupt(void)  
{  
    button_status = 1;  
}
```



Example #2: Global Variable Setting



The image shows a screenshot of an IDE with two panels. The left panel, titled 'C++ source #1', displays the following C++ code:

```
1 int button_status;  
2  
3 void ISR_PinInterrupt(void)  
4 {  
5     button_status = 1;  
6 }
```

The right panel, titled 'ARM GCC 14.1.0 (unknown-eabi) (Editor #1)', displays the corresponding ARM assembly code:

```
1 ISR_PinInterrupt():  
2     mov     r2, #1  
3     ldr     r3, .L2  
4     str     r2, [r3]  
5     bx     lr  
6 .L2:  
7     .word   .LANCHOR0  
8     .set    .LANCHOR0, . + 0  
9 button_status:  
10    .space  4
```


Example #2: Global Variable Setting

```
1  int button_status;
2
3  void wait_for_button_press(void)
4  {
5      button_status = 0;
6      while(button_status == 0){
7          ;
8      }
9  }
10
11
12 void ISR_PinInterrupt(void)
13 {
14     button_status = 1;
15 }
```

```
ARM GCC 14.1.0 (unknown) -Os
1  wait_for_button_press():
2      mov     r2, #0
3      ldr     r3, .L3
4      str     r2, [r3]
5
6      .L2:
7          b     .L2
8
9      .L3:
10         .word .LANCHOR0
11
12 ISR_PinInterrupt():
13     mov     r2, #1
14     ldr     r3, .L6
15     str     r2, [r3]
16     bx      lr
17
18     .L6:
19         .word .LANCHOR0
20         .set  .LANCHOR0, . + 0
21
22 button_status:
23     .space  4
```

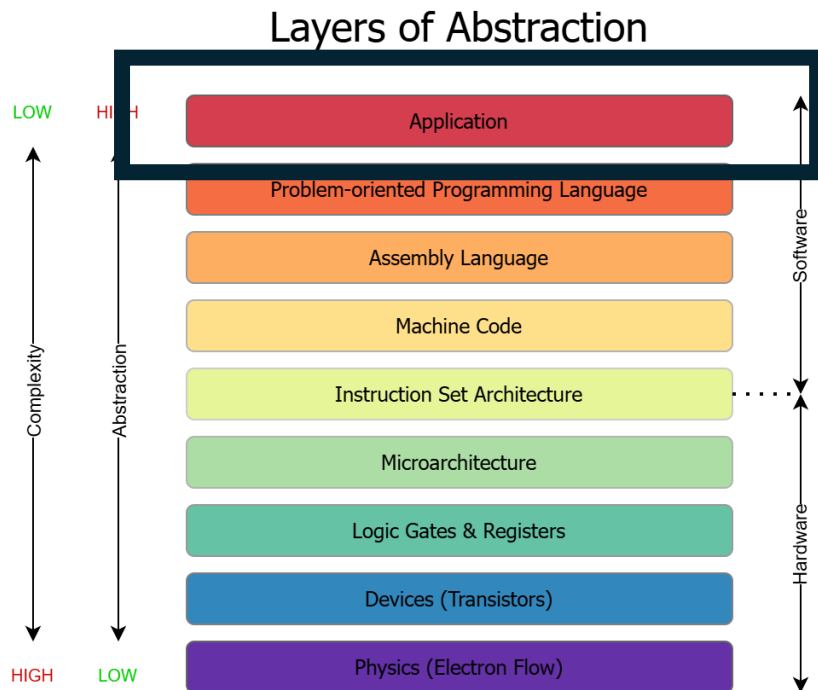
Line 6 on the right is an infinite loop!
b .L2 means “**Branch to Label .L2**”

What was I demonstrating?

The compiler took the High Level Language and made *assumptions*.

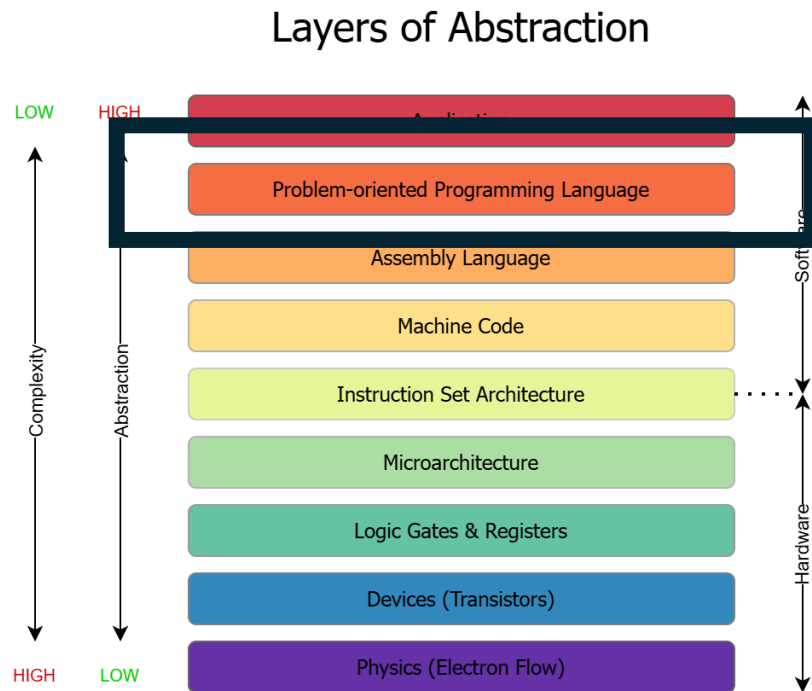
The engineer using the High Level Language *didn't understand what the compiler is going to do*.

Application



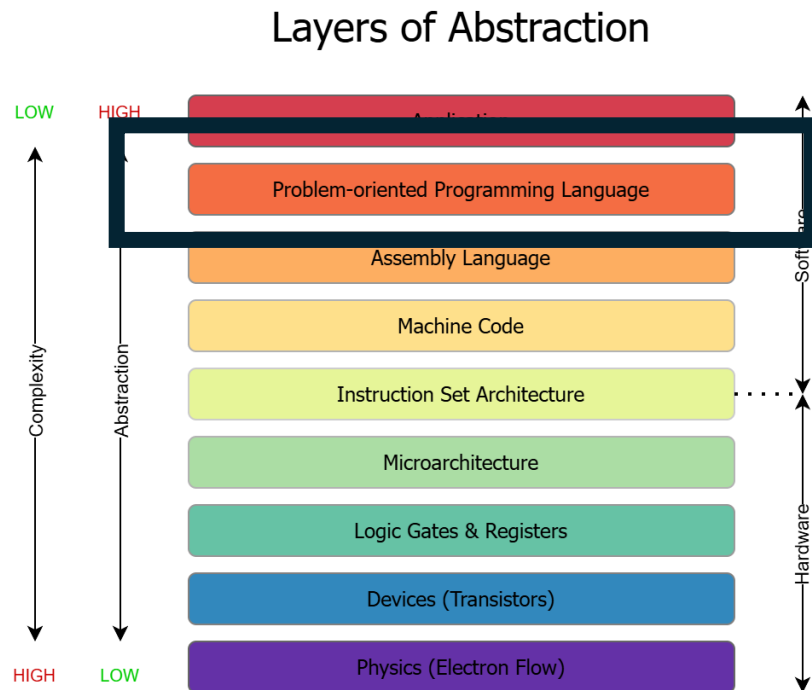
- This is your design intent. This is the most important step in your design.
- We won't cover this in this course – but this follows from many of your design courses.
- Define the problem, constraints, etc.

High Level Language



- This is what most people think of as “programming”
- This can be subdivided further – but consider it could mean:
 - “Bare Metal” C code
 - An RTOS or framework (like FreeRTOS or Arduino)
 - Languages like Python or Java which *may* be interpreted or compiled

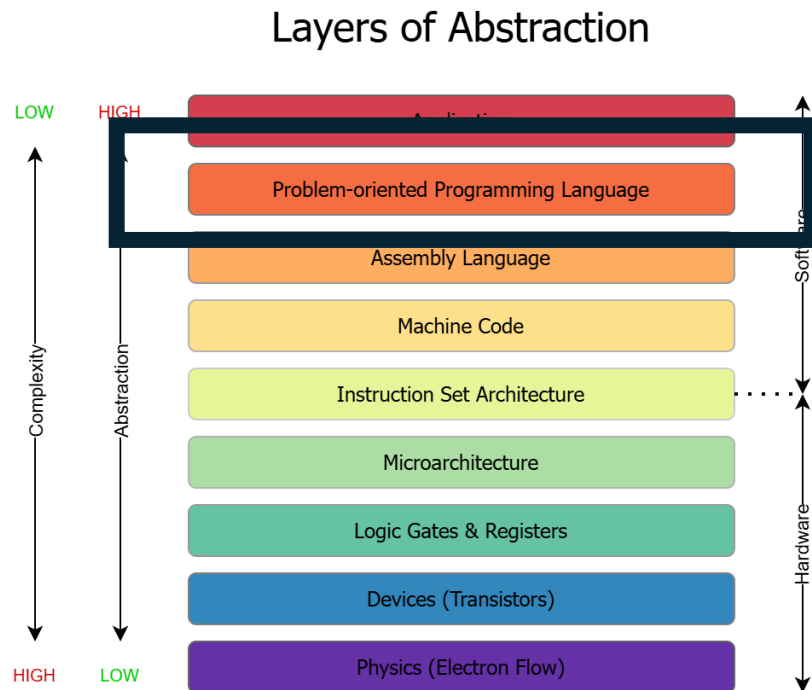
High Level Language



Most Important Embedded Languages (*this is purely my opinion*)

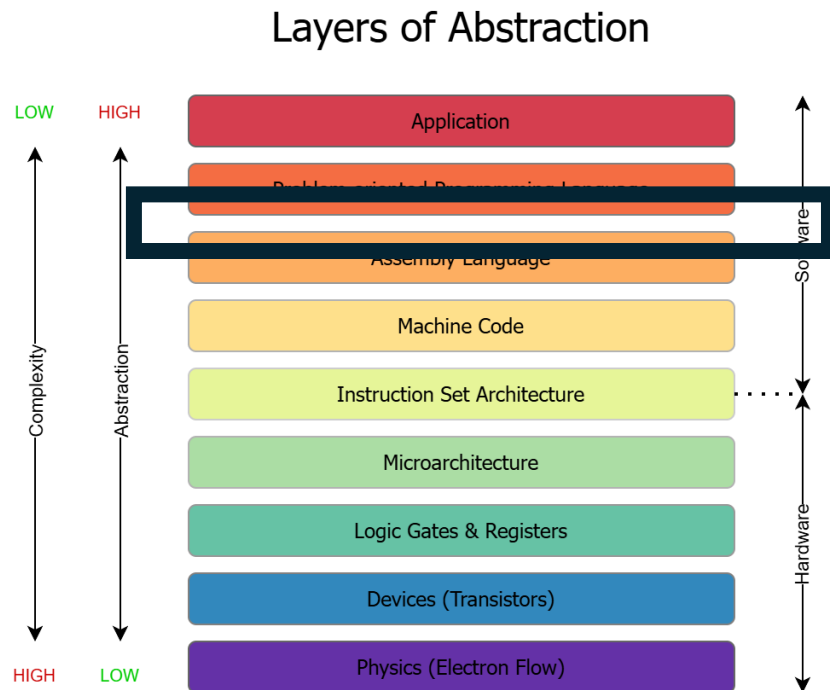
- **C:** *Most common in small embedded systems but used all over.*
- **C++:** Common in larger systems, especially Linux based applications.
- **Java:** Now mostly used for Android development. Many embedded systems run Android however.
- **Rust:** A newer memory-safe language, designed to be usable in place of C. Especially popular among Google as developed there (but used extensively outside as well)

High Level Language



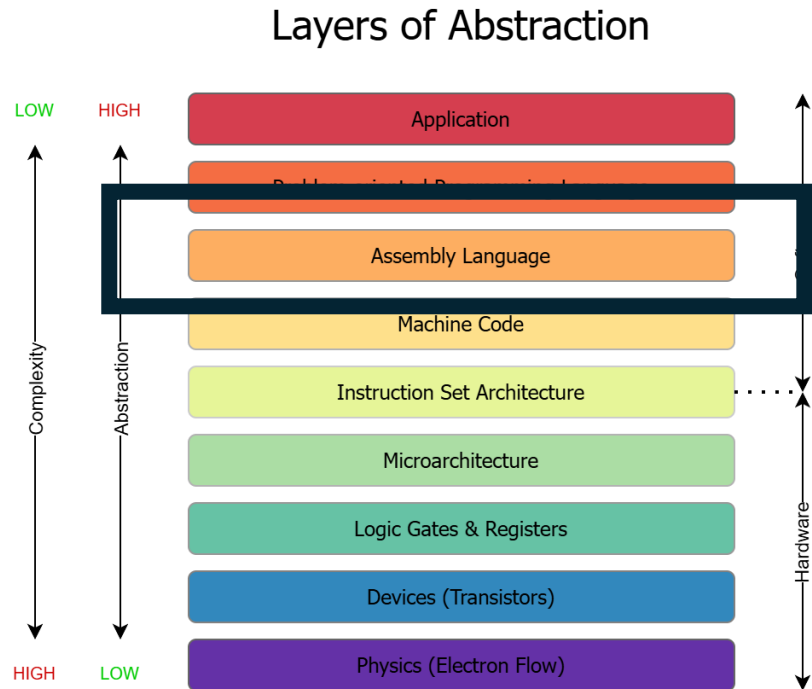
This is the last level at which the code is “machine independent”.

Intermediate Representation



- Compilers have another step that is hidden from us – a language the compiler uses internally for optimizations.
- We'll look at this in more detail in the next lecture.

Assembly Language



- Assembly Language is not machine code.
- Assembly Language is human readable code – remember it is further modified by the Assembler.
- Assembly language is specific to processors. For example, you may see any of the following:
 - Atmel AVR
 - Arm
 - RISC-V
 - x86
 - 8051 (old but still in wide use)
 - MIPS
 - PowerPC
 - Z80 (very uncommon now)

Assembly Example:

```
int check_variable_add(int a, int b) {  
    int c;  
    if (a == 2) {  
        c = a + b;  
    } else {  
        c = a - b;  
    }  
    return c;  
}
```

(NOTE: Following slides have static versions – use [Godbolt.org](https://godbolt.org) and change compiler to see it live)

ARM

```
1  check_variable_add(int, int):  
2      cmp     r0, #2  
3      addeq   r0, r1, #2  
4      subne   r0, r0, r1  
5      bx      lr
```

AVR

```
6  check_variable_add(int, int):  
7  .L__stack_usage = 0  
8      cpi r24,2  
9      cpc r25,__zero_reg__  
10     brne .L2  
11     mov r24,r22  
12     mov r25,r23  
13     adiw r24,2  
14     ret  
15  .L2:  
16     sub r24,r22  
17     sbc r25,r23  
18     ret
```

(Another example of Design Intent vs. Compiler)

My choice of “int” has a specific meaning in C. This forces a longer assembly implementation than if I used an 8-bit “char”.

RISC-V

```
1  check_variable_add(int, int):  
2      li      a5,2  
3      bne     a0,a5,.L2  
4      add     a0,a1,a0  
5      ret  
6  .L2:  
7      sub     a0,a0,a1  
8      ret
```

x86

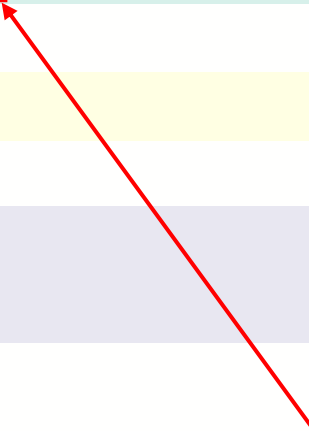
```
1  check_variable_add(int, int):  
2      mov     eax, edi  
3      lea     edx, [rsi+2]  
4      sub     eax, esi  
5      cmp     edi, 2  
6      cmovne  eax, edx  
7      ret
```

MIPS

```
1  check_variable_add(int, int):  
2      li      $3,2                # 0x2  
3      bne     $4,$3,$L4  
4      subu    $2,$4,$5  
5  
6      addiu   $2,$5,2  
7  $L4:  
8      jr      $31  
9      nop
```

MIPS

```
1  check_variable_add(int, int):  
2      li      $3,2                # 0x2  
3      bne     $4,$3,$L4  
4      subu    $2,$4,$5  
5  
6      addiu   $2,$5,2  
7  $L4:  
8      jr      $31  
9      nop
```



MIPS has more fundamentally “out of order” execution. This instruction is always executed.

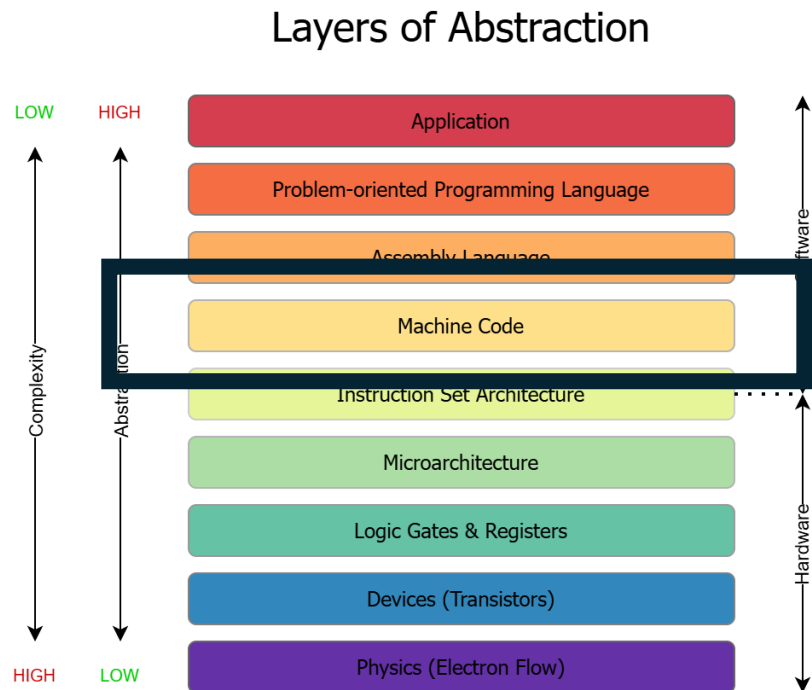
PowerPC

```
1  check_variable_add(int, int):  
2      cmpwi 0,3,2  
3      bne+ 0,.L2  
4      addi 3,4,2  
5      blr  
6  .L2:  
7      subf 3,4,3  
8      blr
```

z80

```
3      public  check_variable_add(int, int).
4  check_variable_add(int, int):
5      call    __frameset0
6      ld      c, (ix + 6)
7      ld      b, (ix + 7)
8      ld      de, 2
9      push    bc
10     pop      iy
11     add      iy, de
12     ld      e, (ix + 4)
13     ld      d, (ix + 5)
14     ld      l, e
15     ld      h, d
16     or       a, a
17     sbc      hl, bc
18     ld      c, l
19     ld      b, h
20     ex       de, hl
21     ld      de, 2
22     or       a, a
23     sbc      hl, de
24     jr       z, BB0_2
25     push     bc
26     pop      iy
27  BB0_2:
28     push     iy
29     pop      hl
30     pop      ix
31     ret
```

Machine Code



- What the machine sees.
- This is the 1's and 0's encoded on "something".
- The machine code normally has some relation to how the code is being interpreted or used.
- The machine ***may still make assumptions about things and does not blindly execute.***

e.g., “No Operation” (NOP)

3.9.8 NOP

No Operation.

Syntax

`NOP{cond}`

where:

- ‘*cond*’ is an optional condition code, see [Conditional execution on page 57](#).

Operation

NOP does nothing. NOP is not necessarily a time-consuming NOP. The processor might remove it from the pipeline before it reaches the execution stage.

Use NOP for padding, for example to place the following instruction on a 64-bit boundary.

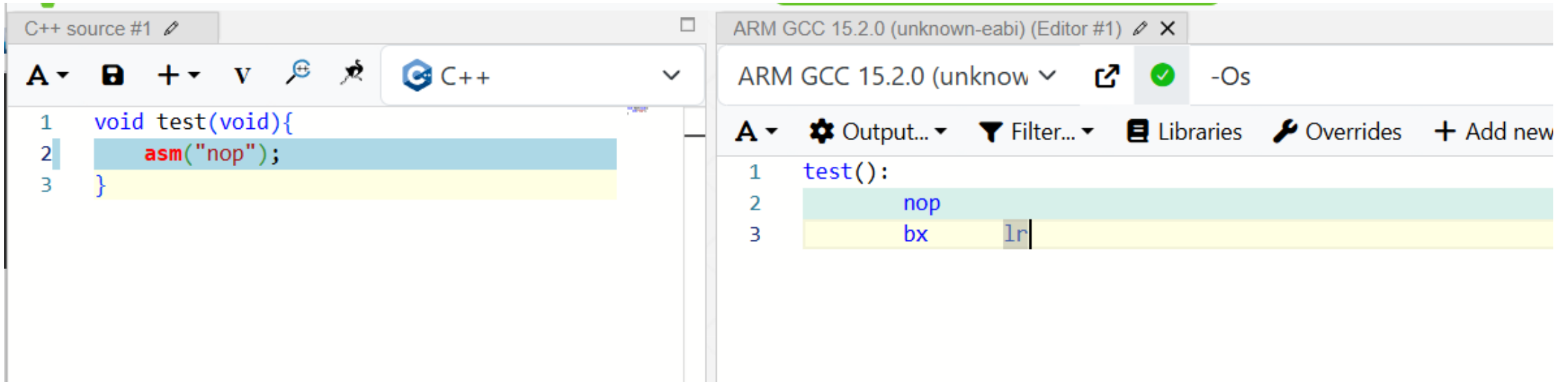
Condition flags

This instruction does not change the flags.

Examples

`NOP ; No operation`

More Assumptions Compiler/Assembler



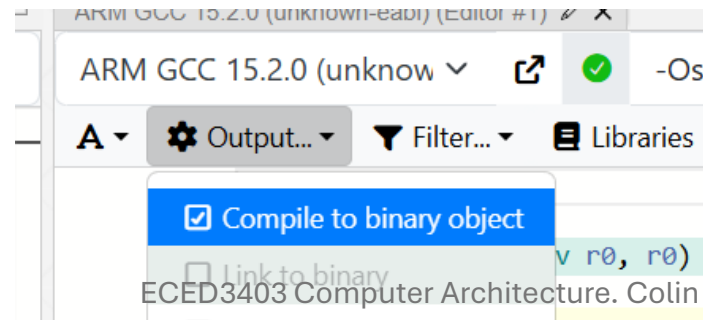
The screenshot shows an IDE with two panels. The left panel, titled 'C++ source #1', contains the following C++ code:

```
1 void test(void){  
2     asm("nop");  
3 }
```

The right panel, titled 'ARM GCC 15.2.0 (unknown-eabi) (Editor #1)', shows the assembly output for the same code. The assembly code is:

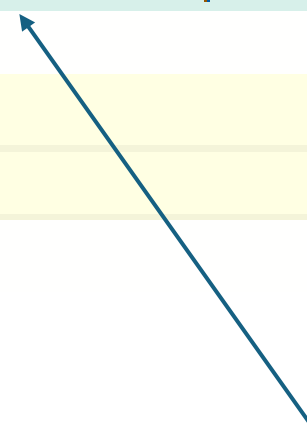
```
1 test():  
2     nop  
3     bx    lr
```

The assembly output is displayed in a light blue background. The right panel also shows a toolbar with options like 'Output...', 'Filter...', 'Libraries', 'Overrides', and 'Add new'.



NOP isn't even used!!!

```
test():  
    e1a00000  
0    nop                @ (mov r0, r0)  
    e12fff1e  
4    bx lr  
4    R_ARM_V4BX *ABS*|
```



This does nothing as well (but isn't the “*nop*” instruction)

Arm Machine Code Example

3	3	2	2	2	2	2	2	2	2	2	2	2	2	1	1	1	1	1	1	1	1	1	9	8	7	6	5	4	3	2	1	0	
1	0	9	8	7	6	5	4	3	2	1	0	9	8	7	6	5	4	3	2	1	0												
Cond	0	0	I											S		Rn		Rd															
Cond	0	0	0	0	0	0	0	A	S							Rd		Rn		Rs		1	0	0	1					Rm			
Cond	0	0	0	0	1	U	A	S								RdHi		RdLo		Rn		1	0	0	1					Rm			
Cond	0	0	0	1	0	B	0	0								Rn		Rd		0	0	0	0	1	0	0	1				Rm		
Cond	0	0	0	1	0	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	0	1					Rn		
Cond	0	0	0	P	U	0	W	L								Rn		Rd		0	0	0	0	1	S	H	1				Rm		
Cond	0	0	0	P	U	1	W	L								Rn		Rd						1	S	H	1				Offset		
Cond	0	1	I	P	U	B	W	L								Rn		Rd															
Cond	0	1	1																														
Cond	1	0	0	P	U	S	W	L								Rn																	
Cond	1	0	1	L																													
Cond	1	1	0	P	U	N	W	L								Rn		CRd		CP#													
Cond	1	1	1	0												CP Opc		CRn		CRd		CP#		CP		0					CRm		
Cond	1	1	1	0												CP Opc	L	CRn		Rd		CP#		CP		1					CRm		
Cond	1	1	1	1																													

332222222222221111111111119876543210

1098765432109876543210

3 3 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1 1 1 9 8 7 6 5 4 3 2 1 0
1 0 9 8 7 6 5 4 3 2 1 0 9 8 7 6 5 4 3 2 1 0

<https://vision.gel.ulaval.ca/~jflalonde/cours/1001/h17/docs/arm-instructionset.pdf>

Arm Machine Code Example

4.4 Branch and Branch with Link (B, BL)

The instruction is only executed if the condition is true. The various conditions are defined [Table 4-2: Condition code summary](#) on page 4-5. The instruction encoding is shown in [Figure 4-3: Branch instructions](#), below.

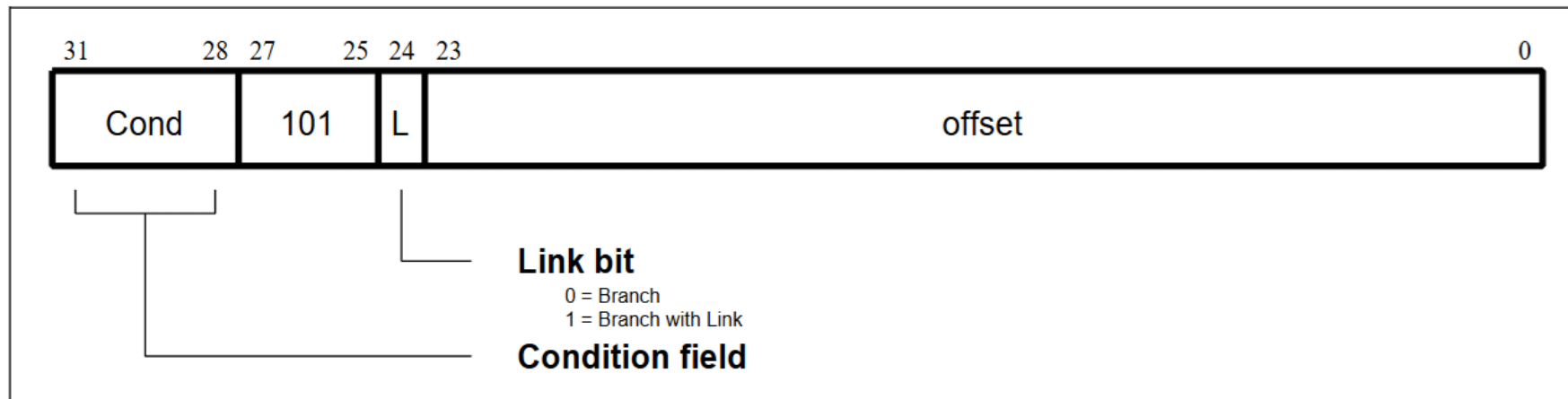
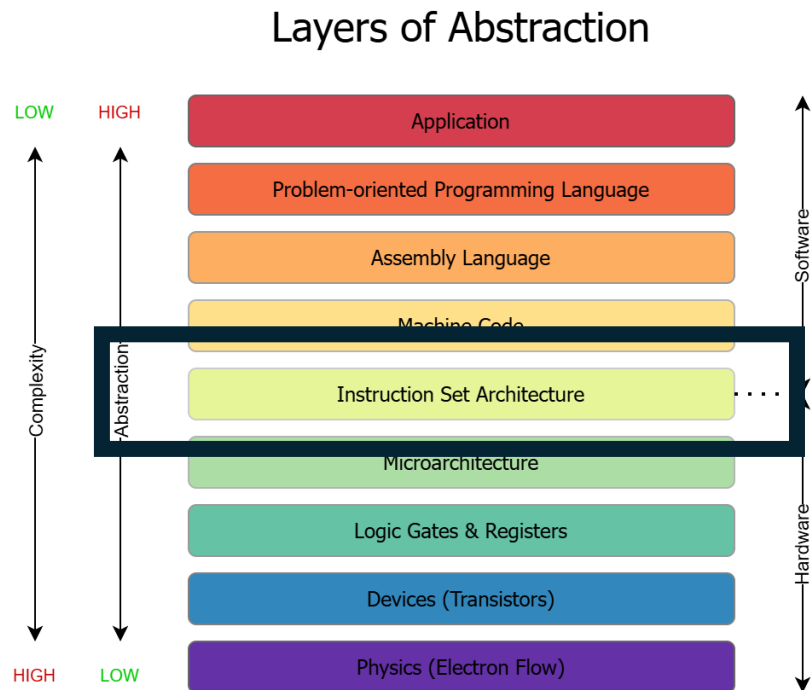


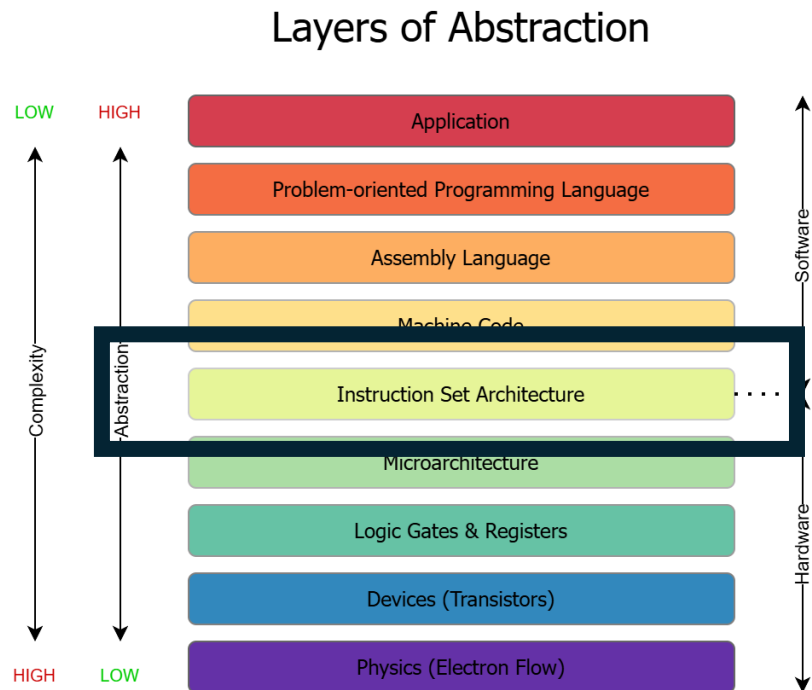
Figure 4-3: Branch instructions

Instruction Set Architecture (ISA)



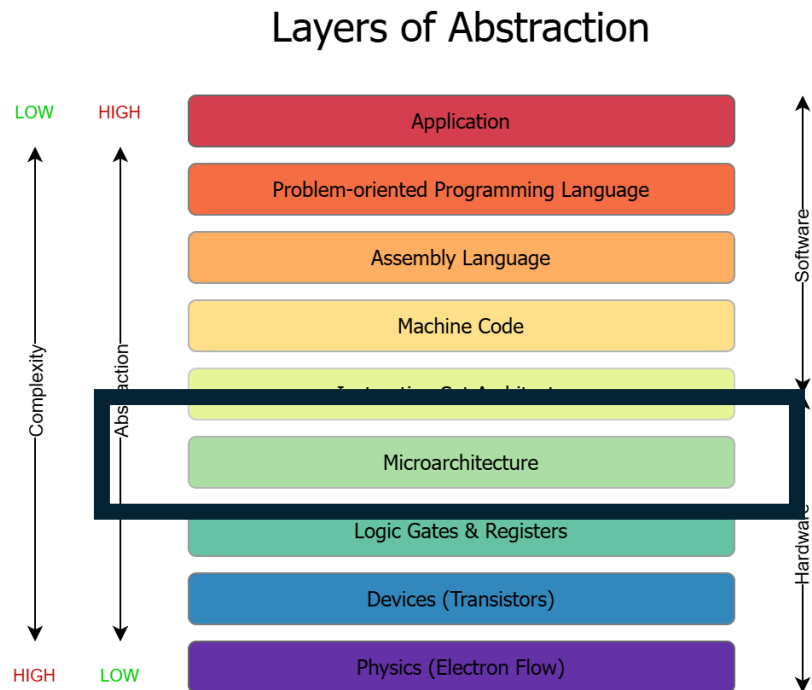
- ISA is actually the specification or design document about how the processor works (the diagram at left could combine machine code & ISA).
- ISA defines:
 - Opcodes
 - Registers
 - Memory models
 - Instruction Encoding
 - ...etc

Instruction Set Architecture (ISA)



- The ISA is a *core part* of this course, we'll be talking about those aspects in great details.
- Some ISA specifications are open-source (RISC-V). This does not mean every implementation of them is Open.
- Non Open-Source mean you cannot even *implement* it without some license agreement (\$\$\$).

Microarchitecture



- This defines how the implementation of the ISA is done.
- The same ISA may have different implementations (microarchitectures).
- Example – your implementation may choose to optimize:
 - **Low-power** (best battery life).
 - Lowest **cost**.
 - Highest **performance** (best speed).

Example of Trade-Offs

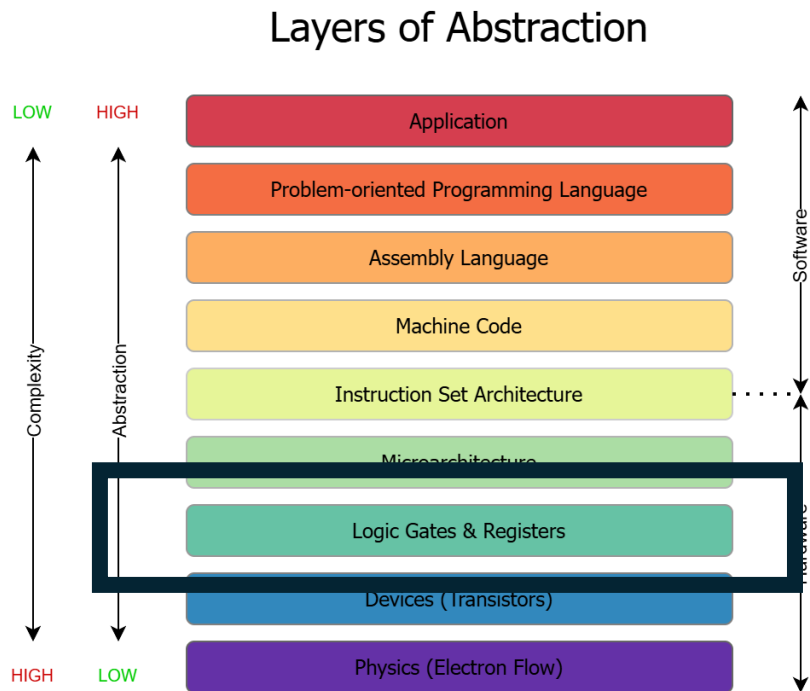
- Low-power & low-cost may have similar goals: minimal area (gates).
 - Re-use logic as much as possible. Serial calculations that may need to wait for resources to be free.
- High-performance
 - Task-specific logic, duplicated logic. Use parallel calculations where possible.

Microcode vs. Hardwired

Advantages / Disadvantages

Hardwired	Microcode
Implemented with fixed logic – cannot be modified or extended.	Implemented with a microprogram – possible to update this to change/add instructions, sometimes even possible "in the field".
Normally faster due to direct hardware usage.	Normally slower do to microinstruction execution.
Complexity/size can increase substantially as more functions needed.	Complexity/size mostly in software – scales well with large changes or for new functions.

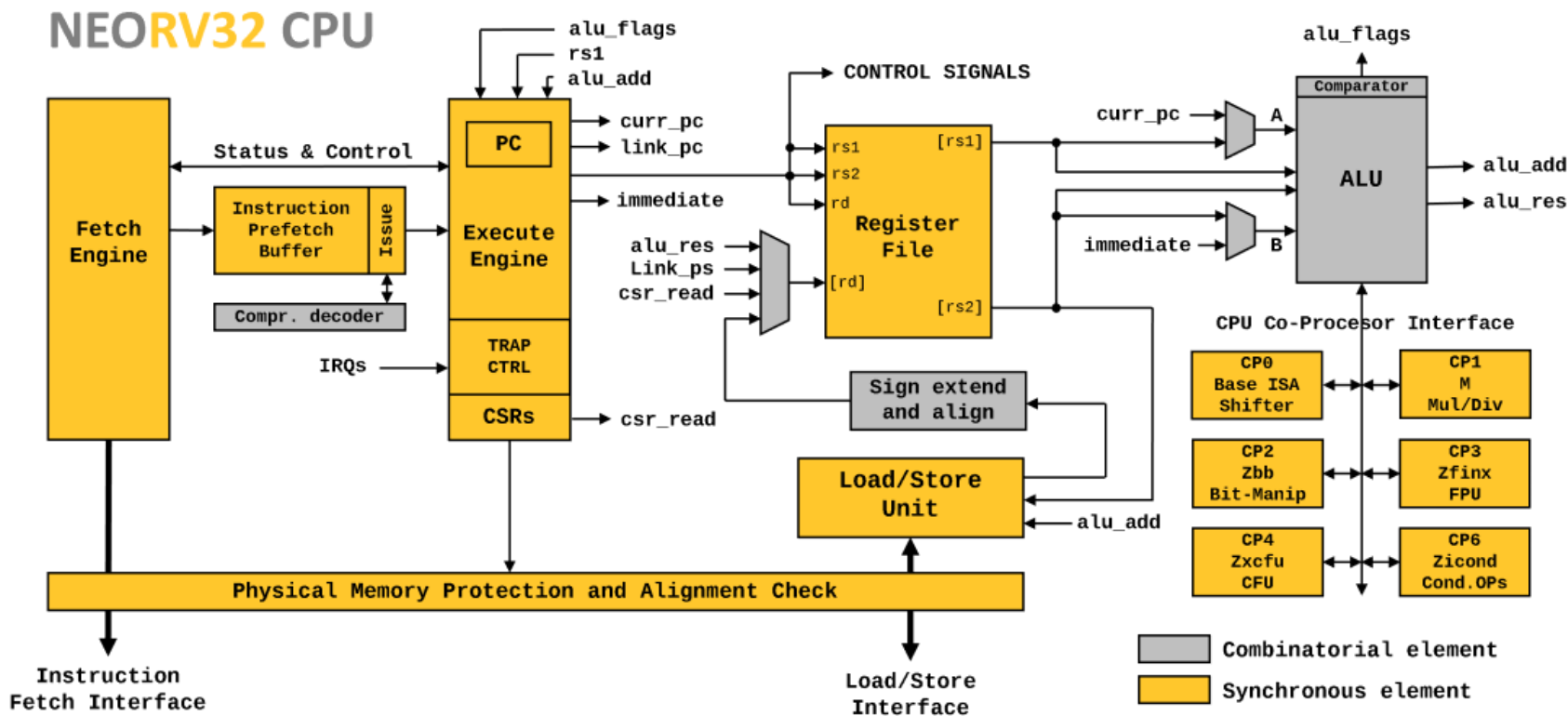
Logic Gates & Registers



- This will be the hardware design itself using gates/flip-flop. Normally defined at the state of "RTL" (**Register-Transfer Level**).
- Examples of RTL are Verilog and VHDL.
- Many blocks are still abstractions at this layer – for example memory (SRAM/FLASH) are defined by the foundry.

RTL Example from neorv32

- Open-source RISC-V Processor, see <https://github.com/stnolting/neorv32>



e.g., ALU Core

https://github.com/stnolting/neorv32/blob/main/rtl/core/neorv32_cpu_alu.vhd

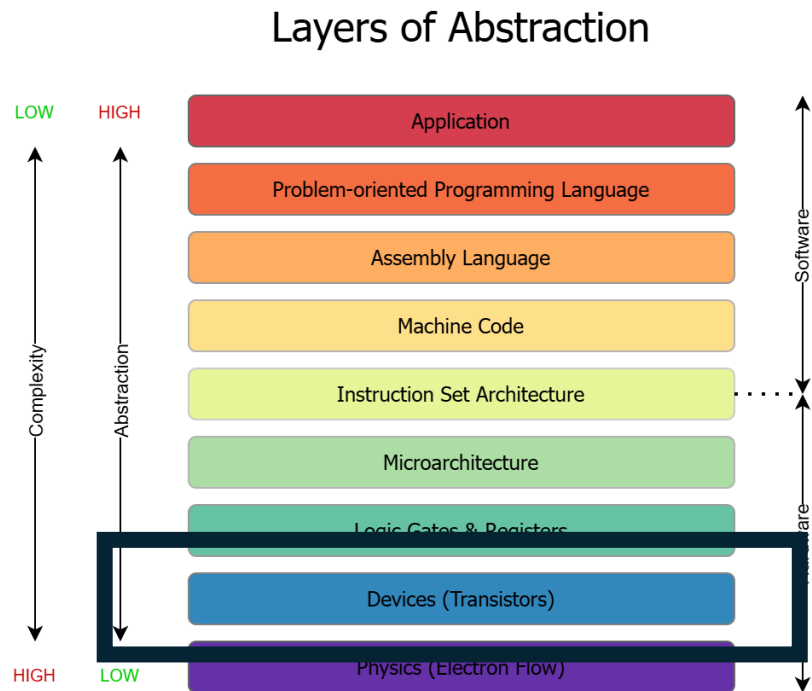
```
-- ALU Core -----  
-----  
  
alu_core: process(ctrl_i, addsub, cp_res, opb, rs1_i)  
begin  
    res_o <= (others => '0');  
    case ctrl_i.alu_op is  
        when alu_op_zero_c => res_o <= (others => '0');  
        when alu_op_add_c  => res_o <= addsub(31 downto 0);  
        when alu_op_cp_c   => res_o <= cp_res;  
        when alu_op_slt_c  => res_o(0) <= addsub(addsub'left); -- carry/borrow  
        when alu_op_movb_c => res_o <= opb;  
        when alu_op_xor_c  => res_o <= opb xor rs1_i;  
        when alu_op_or_c   => res_o <= opb or  rs1_i;  
        when alu_op_and_c  => res_o <= opb and rs1_i;  
        when others       => res_o <= (others => '0');  
    end case;  
end process alu_core;  
  
-- operands --
```

<https://github.com/stnolting/neorv32/tree/main/rtl/core>

RTL Warning / Note

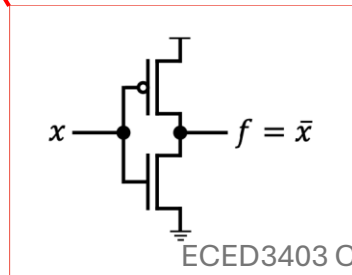
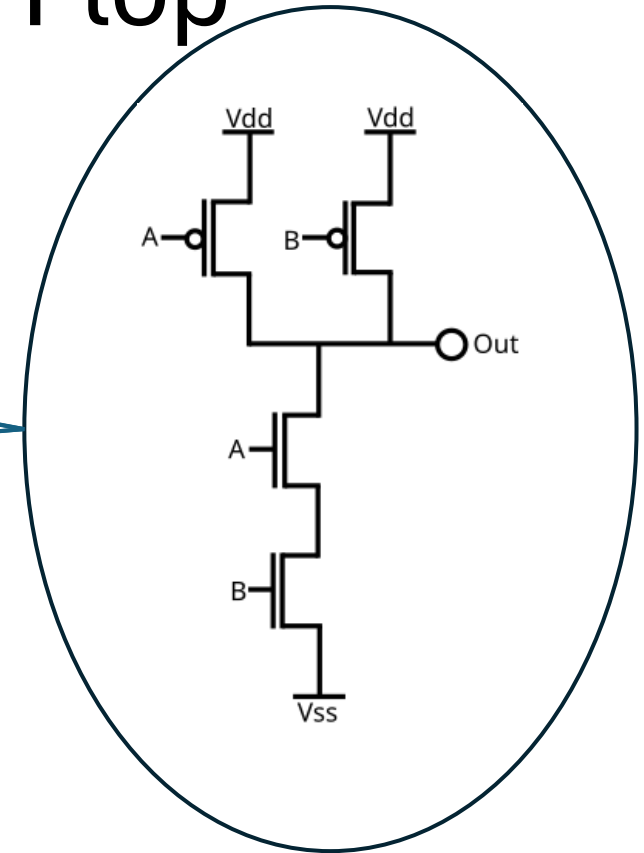
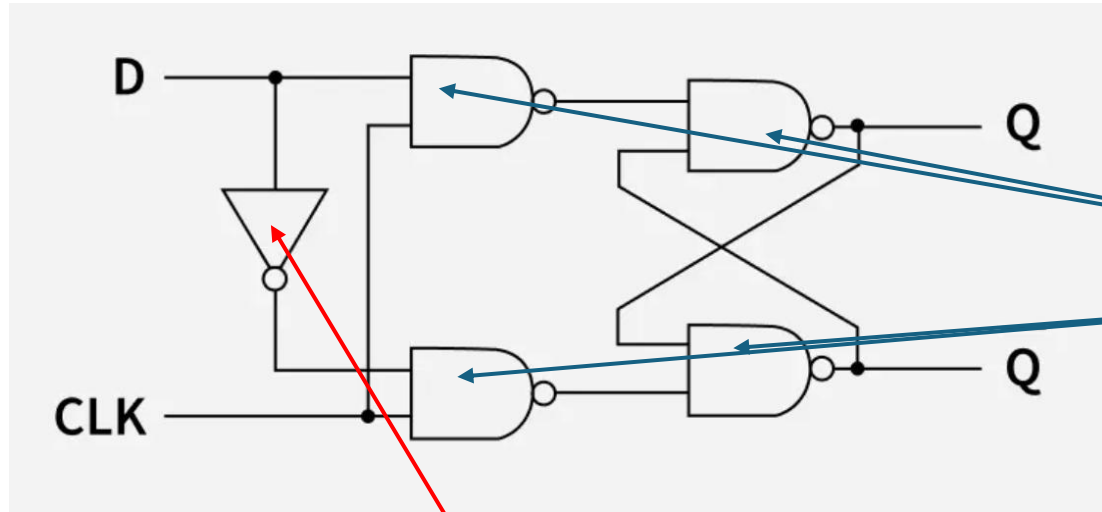
- RTL describes hardware – it looks like software but is not. Everything happens concurrently unless otherwise described elsewhere.
- I don't plan on diving into RTL in this course unless there is a strong interest in it (*Digital Circuits should have introduced it & you'll get more in IC Design if you choose*)

Transistor Level



- The output of the RTL tools goes here – if targeting a FPGA will be based on the gate array. If targeting an ASIC requires additional synthesis steps.

Synthesis Example – 1 bit D Flip-Flop



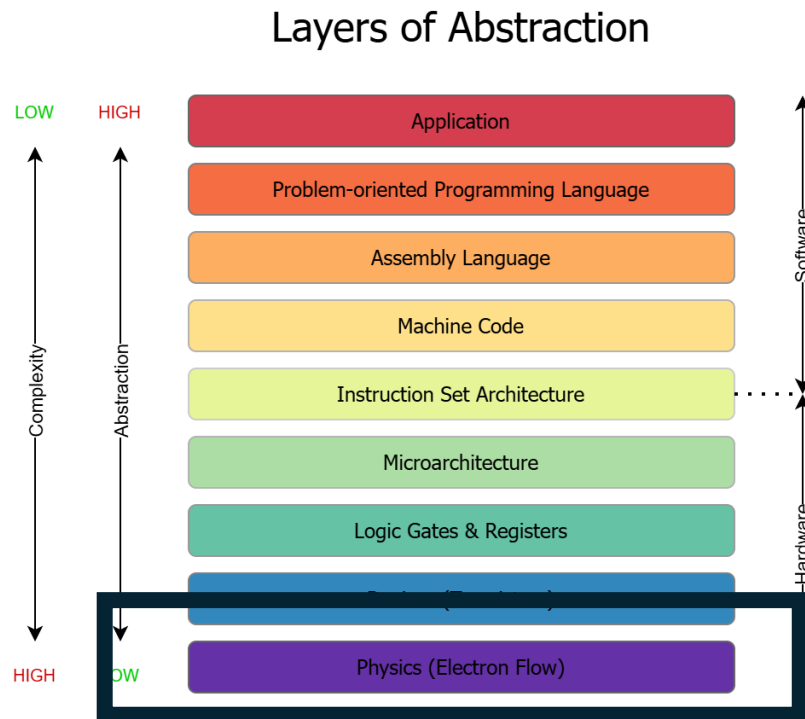
= 18 transistors total!

e.g., Discrete Transistor Level

<https://monster6502.com/>



Implementation Level

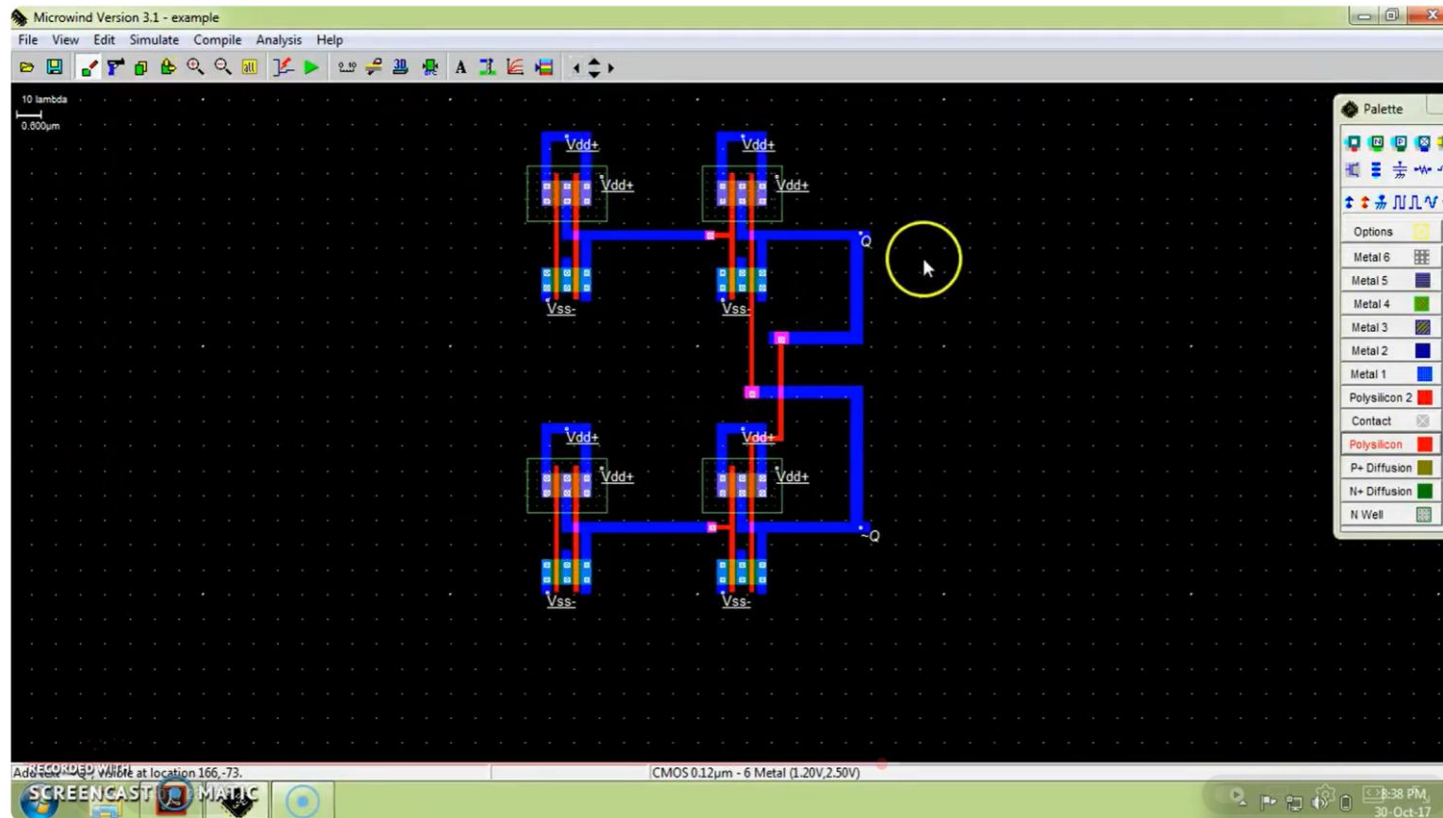


- The output of the RTL tools goes here – if targeting a FPGA will be based on the gate array. If targeting an ASIC requires additional synthesis steps.

Standard Cell Designs

- Most foundries provide *standard cells* – you don't actually synthesize everything to transistor level yourself.
- Allows foundry to optimize for specific process.
- Blocks could include gates, flip-flops, SRAM, etc. Normally also includes *simulation data* required to validate your design.

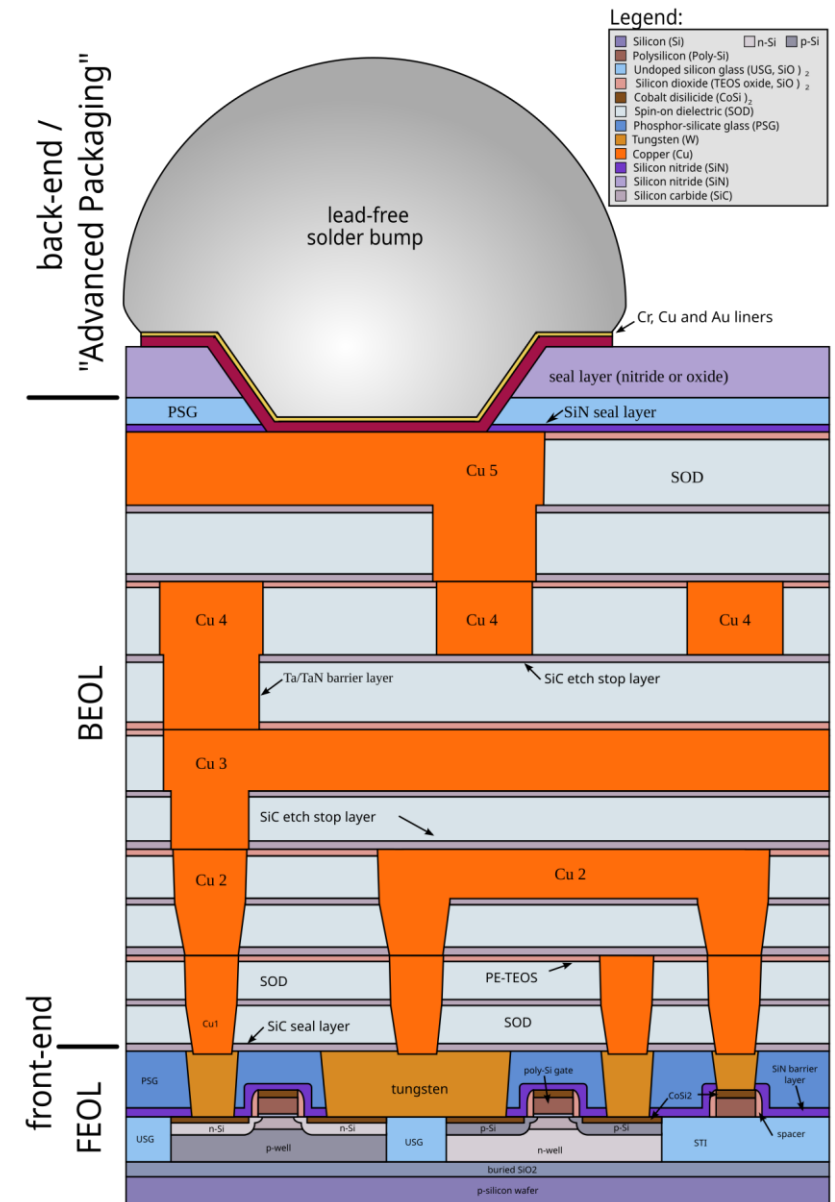
e.g., D FF Layout



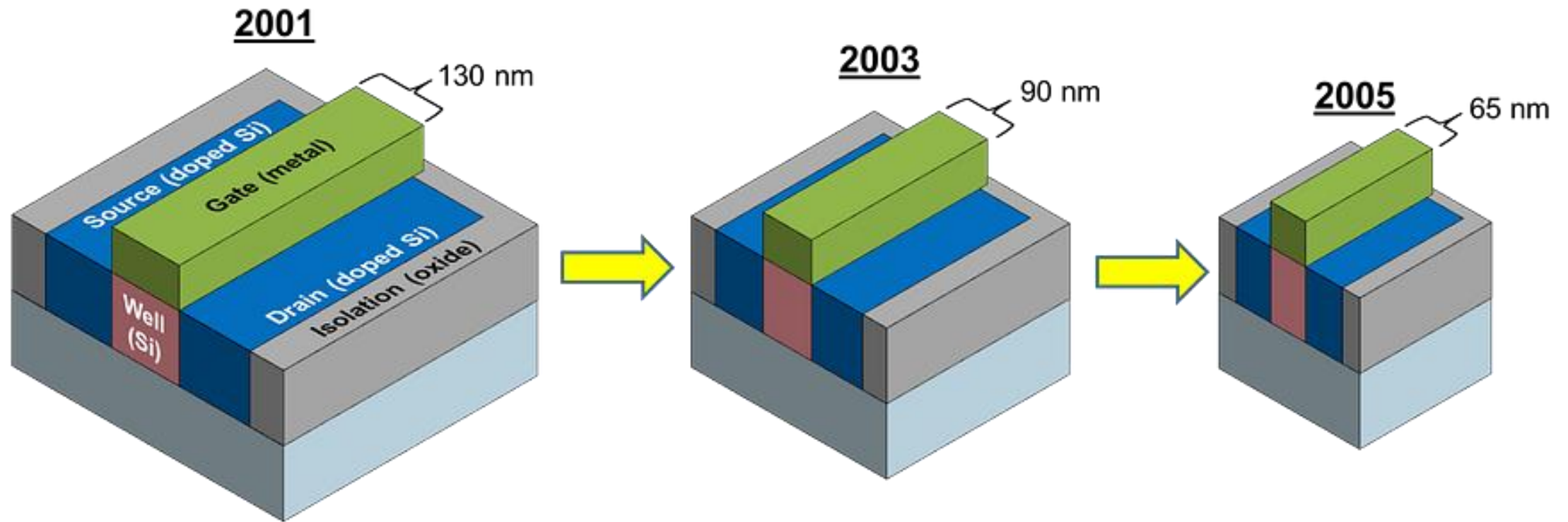
<https://www.youtube.com/watch?v=x8lkfS7IFlw>

Implementation Level

- Specifics of voltage & process are needed at this stage.
- Will define the maximum frequency the device can operate at.



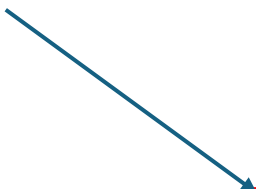
Gate Width?



<https://medium.com/predict/a-node-by-any-other-name-transistor-size-moores-law-b770a16242e5>

Gate Width?

No longer true – note that latest node names are purely marketing.



14 nm process

Article [Talk](#)

[Read](#) [Edit](#) [\](#)

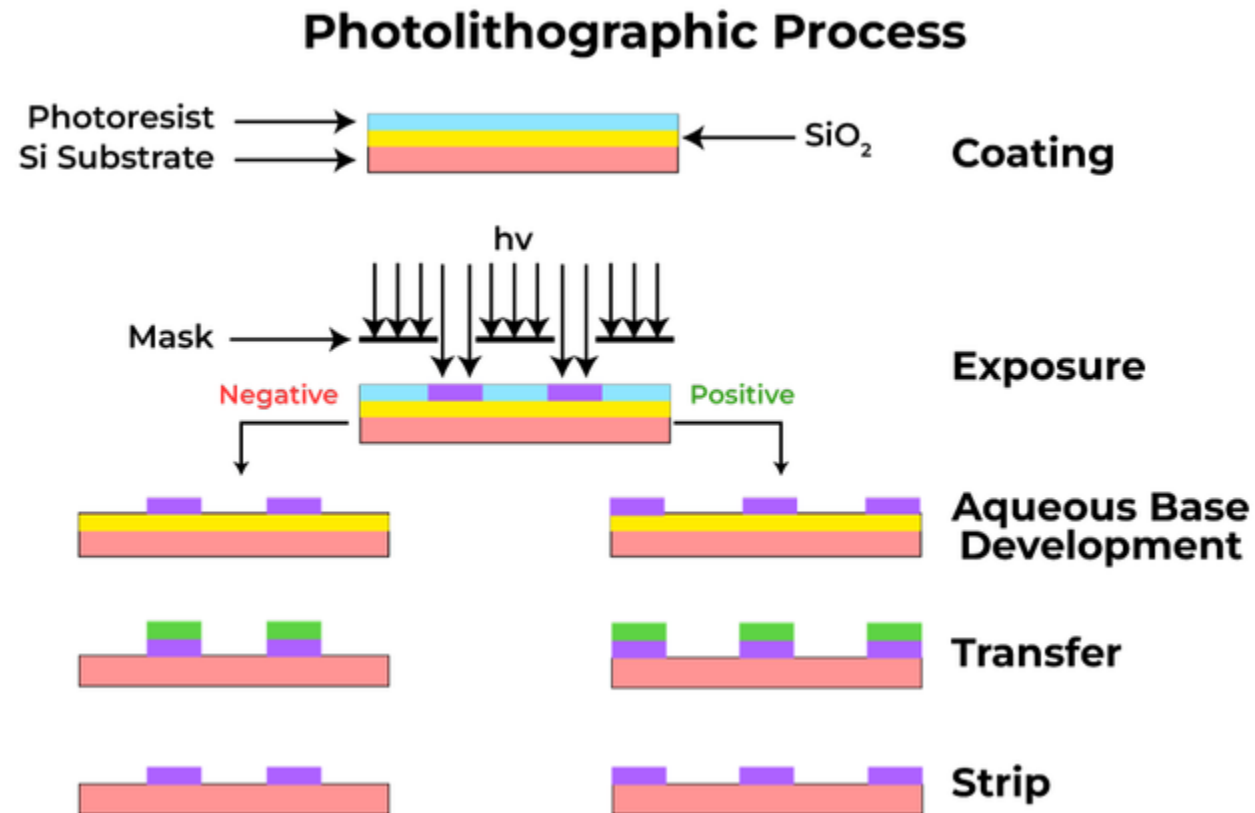
From Wikipedia, the free encyclopedia

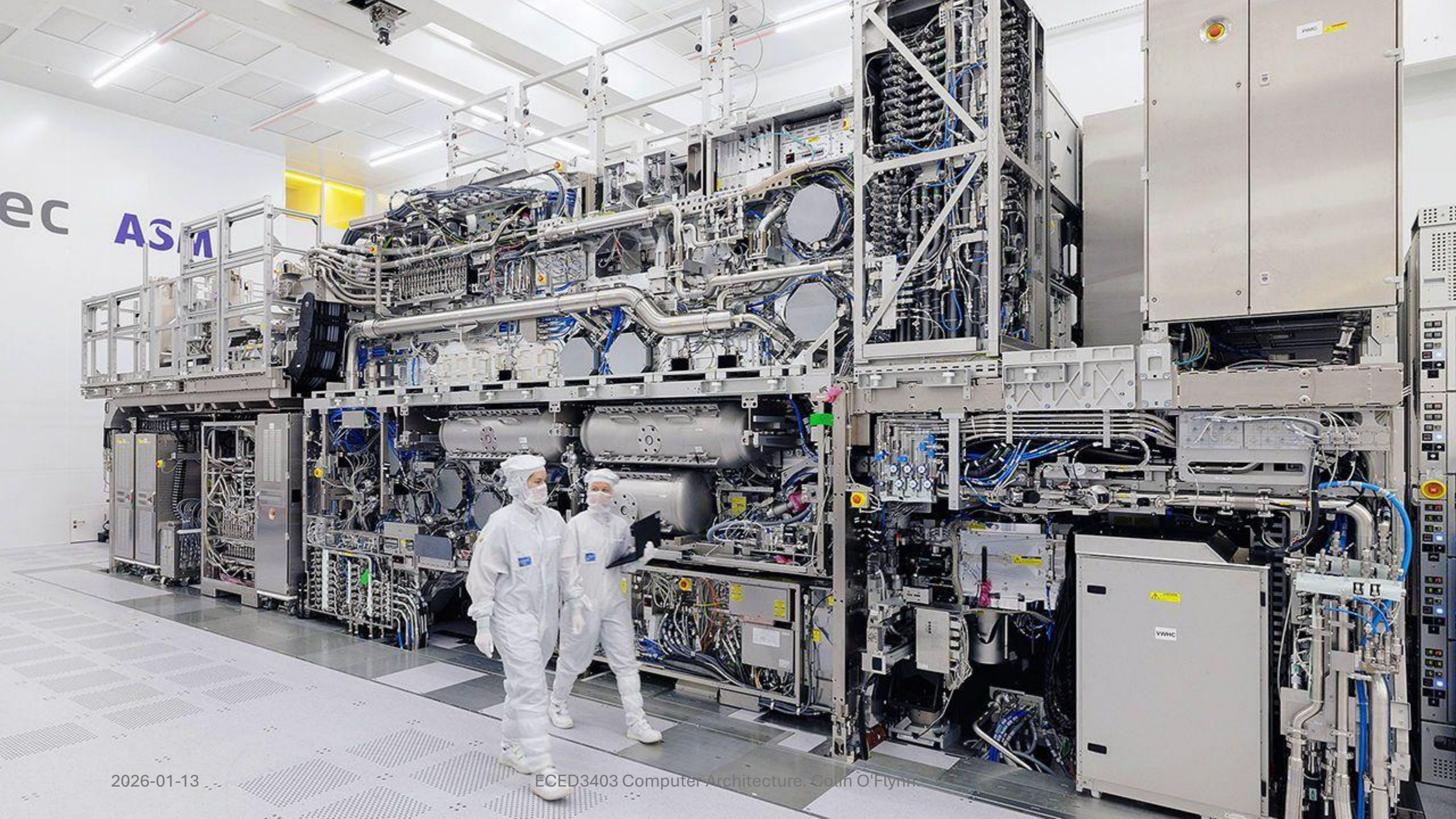
The 12 nm, 14 nm, and 16 nm fabrication nodes are discussed here.

The **14 nanometer process** refers to a marketing term for the [MOSFET technology node](#) that is the successor to the [22 nm](#) (or 20 nm) node. The 14 nm was so named by the [International Technology Roadmap for Semiconductors](#) (ITRS). Until about 2011, the node following 22 nm was expected to be 16 nm. All 14 nm nodes use [FinFET](#) (fin field-effect transistor) technology, a type of [multi-gate MOSFET](#) technology that is a non-planar evolution of planar [silicon CMOS](#) technology.

Since at least 1997, process nodes have been named purely on a marketing basis, and have no relation to the dimensions on the integrated circuit,^[1] neither gate length, metal pitch or gate pitch on a 14nm device is fourteen nanometers.^{[2][3][4]} For example, TSMC and Samsung's 10 nm processes are somewhere between Intel's 14 nm and 10 nm processes in [transistor density](#), and TSMC's [7 nm](#) processes are dimensionally similar to Intel's 10 nm process.^[5]

Making the Chips





ec
ASIA

Layers of Abstraction

